Học thêm, bài tập và hướng dẫn giải

Nguyên lí hệ điều hành — Bài tập mở rộng về tiến trình và đồng bộ

Tài liệu tự biên soạn bổ sung cho 11 bài hiện có, không thay thế bài giảng gốc. Phân biệt mã giả với API cụ thể; ví dụ về đồng bộ dùng mô hình lý thuyết.

Mục tiêu học thêm

- Phân biệt đồng thời, song song, tiến trình và tiểu trình
- Phân tích trạng thái tiến trình và lựa chọn cơ chế liên lạc
- Tìm race condition, deadlock và lỗi thứ tự lấy khóa

1. Đồng thời và song song

Đồng thời nghĩa là nhiều công việc có tiến triển trong các khoảng thời gian chồng lấn; trên một lõi, hệ điều hành có thể xen kẽ chúng. Song song là thực thi tại cùng thời điểm trên nhiều đơn vị xử lý. Có nhiều thread không bảo đảm chương trình nhanh hơn: còn phụ thuộc khối lượng công việc, chờ I/O, tranh chấp khóa và chi phí chuyển ngữ cảnh.

Tiến trình có không gian địa chỉ và tài nguyên riêng theo mô hình hệ điều hành. Các thread cùng tiến trình thường chia sẻ không gian địa chỉ nhưng mỗi thread có stack và trạng thái thực thi riêng. Chia sẻ dữ liệu thuận tiện hơn, đồng thời cũng dễ phát sinh tranh chấp.

2. Trạng thái và lập lịch

Một mô hình đơn giản có ready, running và blocked. Ready là đủ điều kiện chạy nhưng chờ CPU; blocked là đang chờ sự kiện như I/O. Khi hết quantum, tiến trình running có thể quay về ready. Hoàn tất I/O thường đưa blocked về ready, không nhất thiết được chạy ngay.

Ví dụ Round Robin: A cần 5 đơn vị CPU, B cần 3, cùng tới thời điểm 0, quantum=2, hàng đợi ban đầu A rồi B, không có I/O và bỏ qua chi phí chuyển ngữ cảnh.

- A chạy 0–2, còn 3.
- B chạy 2–4, còn 1.
- A chạy 4–6, còn 1.
- B chạy 6–7, hoàn tất.
- A chạy 7–8, hoàn tất.

Turnaround của A là 8, B là 7. Waiting time bằng turnaround trừ tổng CPU trong mô hình này: A=3, B=4. Nếu có I/O thì không áp dụng máy móc công thức trừ CPU mà bỏ qua thời gian bị blocked.

3. Lựa chọn IPC

Pipe phù hợp luồng dữ liệu giữa các tiến trình trong tình huống thích hợp trên cùng máy; socket phù hợp liên lạc qua mạng hoặc cùng máy. Shared memory cho phép chia sẻ vùng dữ liệu nhưng cần tự giải quyết đồng bộ và quyền truy cập. Message passing giúp tách vùng nhớ, nhưng phải xem xét kích thước thông điệp, sao chép và việc gửi/nhận có chặn hay không.

Không chọn IPC chỉ vì "nhanh nhất". Cần biết các tiến trình có cùng máy không, có quan hệ cha/con không, dữ liệu lớn bao nhiêu, có bao nhiêu bên gửi/nhận và cần xử lý lỗi bên kia dừng thế nào.

4. Race condition

Phép counter = counter + 1 có thể gồm đọc, tính và ghi, không nên mặc định nguyên tử. Hai thread cùng đọc 0, cùng tính 1 rồi cùng ghi 1 làm mất một lần tăng; kết quả đúng mong đợi là 2.

Mutex bảo vệ vùng đọc-sửa-ghi; mọi đường truy cập dữ liệu chia sẻ cần tuân thủ cùng quy tắc khóa. Không chỉ khóa dòng ghi cuối vì hai thread vẫn có thể tính từ dữ liệu cũ. Cần bảo đảm khóa được giải phóng khi có lỗi.

5. Producer–consumer và thứ tự semaphore

Với buffer có N ô, đặt empty=N, full=0 và mutex=1. wait giảm semaphore hoặc chờ khi không thể giảm; signal tăng và có thể đánh thức bên chờ.

```
Producer:
  wait(empty)
  wait(mutex)
  thêm dữ liệu vào buffer
  signal(mutex)
  signal(full)

Consumer:
  wait(full)
  wait(mutex)
  lấy dữ liệu khỏi buffer
  signal(mutex)
  signal(empty)
```

Không lấy mutex rồi mới chờ empty/full: nếu giữ mutex trong khi buffer đầy/rỗng, bên cần thay đổi buffer có thể không lấy được mutex để giải phóng điều kiện chờ. Với condition variable, thường kiểm tra điều kiện trong vòng while dưới mutex và dùng thao tác chờ phù hợp giải phóng mutex; không dùng if một lần rồi giả định điều kiện luôn còn đúng khi thức dậy.

6. Deadlock và starvation

Deadlock là các bên không thể tiếp tục do chờ tài nguyên nhau. Ví dụ T1 giữ A chờ B, T2 giữ B chờ A. Quy định mọi thread lấy A trước B loại vòng chờ này. Starvation là một bên liên tục không được phục vụ dù hệ thống vẫn tiến triển; không đồng nghĩa với deadlock. Một khóa có mutual exclusion chưa tự bảo đảm fairness.

Bài tập và gợi ý đáp án

1. Dễ: I/O hoàn tất có nghĩa tiến trình chạy ngay không? Không: thông thường chuyển về ready rồi chờ lập lịch.
2. Trung bình: lập bảng Round Robin ở ví dụ và tính thời điểm phản hồi đầu tiên. A=0, B=2; kiểm tra tổng CPU là 8.
3. Khó: producer giữ mutex rồi chờ empty khi buffer đầy. Consumer cần mutex để lấy phần tử nên không giải phóng empty được; đổi thứ tự như mã giả trên. Phân tích thêm lỗi nếu quên signal(mutex).

Tiêu chí và nguồn học tiếp

Vẽ timeline xen kẽ từng thao tác, ghi trạng thái biến và giá trị semaphore; không chỉ kết luận "có lỗi". Với lời giải đồng bộ, chỉ ra vùng tới hạn, điều kiện chờ và bên có thể làm điều kiện thay đổi. Với lập lịch, luôn ghi giả định về thời điểm đến, quantum và chi phí chuyển ngữ cảnh.

https://pages.cs.wisc.edu/~remzi/OSTEP/ ; các chương CPU Scheduling, Threads, Locks, Condition Variables và Semaphores.